

Data Preparation — Practice Problems

Instructor: Dr. Akash Gupta

Table of contents

Part 1: Questions	2
Part 2: Answers and Explanations	23

Part 1: Questions

Question 1

A junior analyst at Capital One is in a hurry and skips data preparation, feeding the raw CSV directly into `LogisticRegression()`. The fit fails with errors and warnings. Which statement BEST explains why data preparation is essential — not optional?

- a) Scikit-learn refuses to train any model unless data is preprocessed
- b) Real-world tabular data almost always contains issues (missing values, mixed types, wildly different scales, duplicates, inconsistent encodings) that mathematical models cannot consume directly; the preparation step is what turns business data into a model-ready matrix
- c) Data preparation is a performance optimization that improves runtime
- d) Preparation is required only for classification, not regression
- e) Preparation is the same as feature engineering and can be skipped if no new features are needed

[☐ Jump to solution](#)

Question 2

In the CRISP-DM workflow, where does **data preparation** sit?

- a) Before Business Understanding
- b) CRISP-DM does not have a data-preparation stage
- c) After Deployment
- d) It runs in parallel with Evaluation
- e) Between Data Understanding and Modeling — typically the most time-consuming stage of a real project

[☐ Jump to solution](#)

Question 3

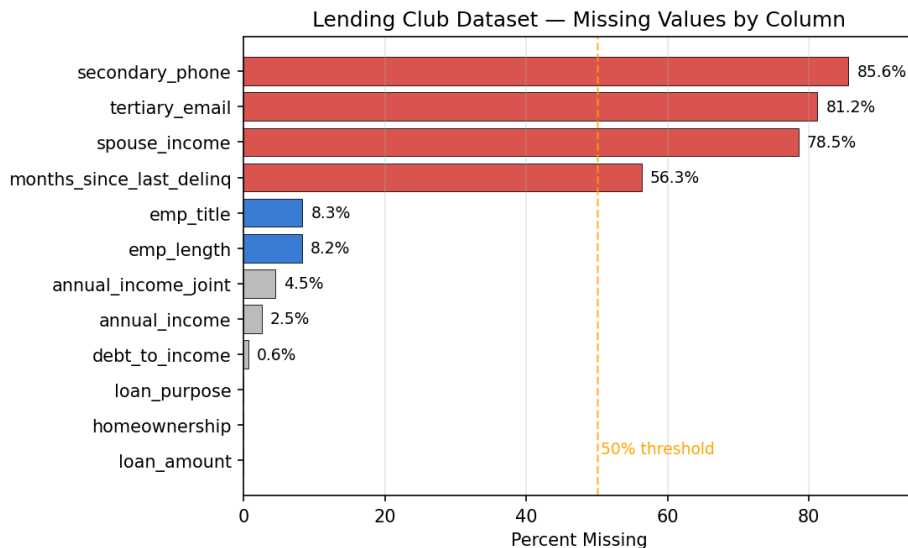
A Capital One credit dataset has missing values in the `income` column. The likelihood of a row's income being missing depends on the borrower's actual income (very high earners often skip it). Which type of missingness is this?

- a) MCAR (Missing Completely At Random)
- b) MNAR (Missing Not At Random) — the missingness is informative; ignoring it can bias the model
- c) Censored data
- d) MAR (Missing At Random)
- e) Aggregated missingness

[☐ Jump to solution](#)

Question 4

A Lending Club analyst inspects a new dataset and produces this missing-value bar chart:



The analyst now wants the underlying numbers: a one-line pandas summary of how much is missing per column. Which snippet does this?

- a) `df.fillna(0)`
- b) `df.dropna(axis=0)`
- c) `df.isna().sum()`
- d) `df.describe()`
- e) `df.columns.missing()`

[□ Jump to solution](#)

Question 5

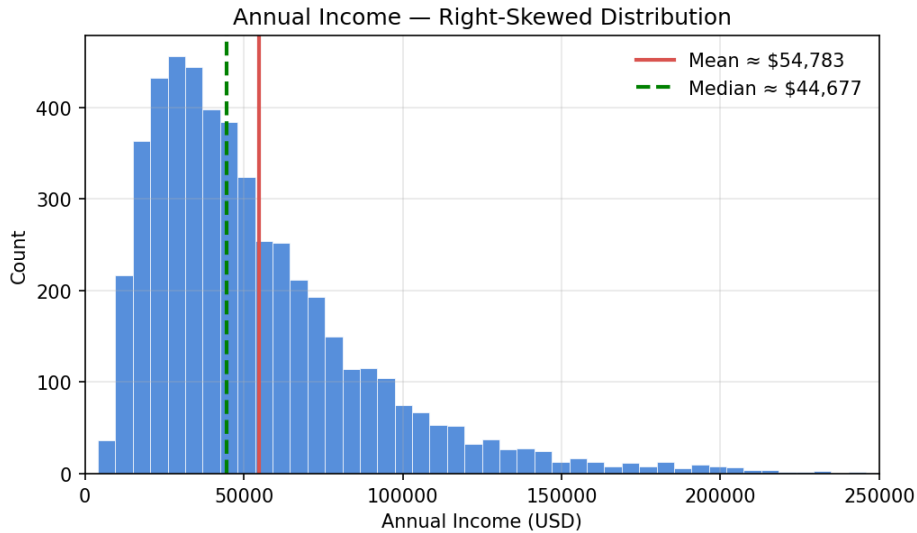
A column `secondary_phone` is missing for 87% of rows in a customer-churn dataset. Which approach is most defensible?

- a) Mean-impute the column
- b) Replace missing with 0
- c) Drop the column — with that little signal remaining, imputation introduces noise far larger than any predictive value the column could carry; flag the decision in the data dictionary
- d) Replace missing with the column's mode
- e) Use the most-recent customer's phone number

[□ Jump to solution](#)

Question 6

The `annual_income` column has 8% missing. Its histogram (with mean and median marked) is shown below.



Which simple-imputation strategy is MOST appropriate to fill the missing 8%?

- a) Mean — preserves the average
- b) Use `random.uniform(min, max)` to draw a value
- c) Drop the column
- d) Replace with zero
- e) Median — robust to skew; mean would be pulled up by the high-earner tail and bias every imputed row toward an unrealistic middle

[□ Jump to solution](#)

Question 7

The `home_ownership` column is categorical (RENT / OWN / MORTGAGE) and has 3% missing. Which simple imputation is appropriate?

- a) Mode — the most frequent category; mean and median are not defined for non-numeric categorical variables
- b) Median
- c) Mean
- d) Drop the row
- e) Replace with the empty string

[□ Jump to solution](#)

Question 8

A team's small dataset has missing values in `monthly_spend` that depend strongly on other features (age, region, plan). Which advanced imputer would likely produce more realistic values than mean / median?

- a) KNN imputation — fills missing values using the average of the K most similar rows on the non-missing features
- b) Replace with NaN
- c) Drop the row
- d) Bayesian network simulation
- e) Bootstrapping

[☐ Jump to solution](#)

Question 9

What does `IterativeImputer` (the MICE-style imputer in scikit-learn) do?

- a) Drops every row containing any missing value
- b) Iteratively models each column with missing values as a function of the others, refining estimates over several passes — useful when many columns are missing and missingness has structure
- c) Performs label encoding before any model is trained
- d) Replaces missing values with the column mean only
- e) Identical to mean imputation

[☐ Jump to solution](#)

Question 10

Sometimes whether a value is missing is itself informative (e.g., `time_since_last_delinquency` is missing precisely for customers who never had a delinquency). What is a clean, low-risk way to capture this signal?

- a) Drop the column
- b) Add a binary "missingness indicator" column (1 if the original value was missing, 0 otherwise) and then impute the original column with median or zero. The model can use the indicator to learn that missingness itself predicts the target
- c) Set missing to zero and ignore
- d) Replace missing with -999
- e) Train a separate model only on rows with missing values

[☐ Jump to solution](#)

Question 11

A student writes:

```
imputer = SimpleImputer(strategy="median")
X_imputed = imputer.fit_transform(X)          # uses ALL rows
X_train, X_test, y_train, y_test = train_test_split(
    X_imputed, y, test_size=0.2, random_state=42)
```

What is the bug?

- a) SimpleImputer cannot use the median
- b) The imputer is fit on the full x before the split, so the test rows contribute to the median used to fill training rows. The test set is no longer truly held out and the reported test metric is optimistically biased. Fix: split first, fit the imputer on training only — typically inside a Pipeline
- c) random_state=42 is illegal
- d) train_test_split rejects imputed data
- e) The strategy must be "mean"

[□ Jump to solution](#)

Question 12

Compared with simple median imputation, multivariate imputation (KNN, MICE) is more accurate but has TRADEOFFS. Which is the BEST description of the cost?

- a) It is slower, more complex, harder to debug, more sensitive to outliers and feature scaling, and risks introducing leakage if implemented carelessly across a train / test split
- b) It cannot be combined with logistic regression
- c) It is unavailable in scikit-learn
- d) It is mathematically incorrect
- e) It always produces worse results than median imputation

[□ Jump to solution](#)

Question 13

A Shopify orders dataset has the same order recorded twice because of a webhook glitch. Which pandas call removes exact duplicate rows?

- a) `df.fillna(method="ffill")`
- b) `df.dropna()`
- c) `df.unique()`
- d) `df.drop_duplicates()`

e) `df.merge(df)`

[□ Jump to solution](#)

Question 14

A Visa fraud dataset has a heavily right-skewed `transaction_amount` column. A junior analyst flags outliers using **z-score > 3**, but a senior teammate prefers the **IQR rule**. Why?

- a) Z-score is faster
- b) Z-score uses the mean and standard deviation, BOTH of which are themselves inflated by extreme values; on heavy-tailed data the inflated standard deviation can hide most outliers. The IQR uses quartiles, which are robust to extremes
- c) Z-score is a deprecated method
- d) Z-score requires the column to be standardized first
- e) IQR works only for binary classification

[□ Jump to solution](#)

Question 15

When does **capping** (winsorizing) outliers — clipping any value above the 99th percentile to the 99th percentile — make sense?

- a) When you want to delete those rows entirely
- b) When the column is categorical
- c) When the extreme values are real but their numeric magnitude distorts a model that assumes a roughly bounded scale (like a linear regression with regularization), and you want to keep the row's other information without letting one column dominate
- d) Capping is never appropriate
- e) Only for the target column, never for features

[□ Jump to solution](#)

Question 16

A `home_price` feature is right-skewed (a long tail of expensive homes). Which transformation often makes the feature more linearly related to the target and stabilizes variance?

- a) Z-score normalization
- b) One-hot encoding
- c) Polynomial features
- d) Log transformation — `np.log1p(price)` compresses the long tail and turns multiplicative effects into additive ones, often improving the fit of linear models
- e) Identity (do nothing)

[□ Jump to solution](#)

Question 17

A retailer's sales dataset has a few \$50,000 days during Black Friday that look like "outliers" by IQR. The team is tempted to drop them. What is the BEST recommendation?

- a) Drop them — they're outliers
- b) Cap them at the 95th percentile of normal days
- c) Replace with the median
- d) Convert them to log scale and forget about it
- e) Keep them — Black Friday spikes are real signal, not noise. Dropping or capping them would force the model to ignore one of the most important business patterns. Consider a feature `is_black_friday` so the model can use the spike

[☐ Jump to solution](#)

Question 18

Which encoding is most appropriate for a `state` column with all 50 US states?

- a) Z-score normalization
- b) Label encoding 1–50 (state alphabetical order)
- c) One-hot encoding — states have no natural order; assigning integers 1–50 would imply $CA < NY < TX$, which is meaningless
- d) Drop the column
- e) Random integers between 0 and 100

[☐ Jump to solution](#)

Question 19

Which encoding is most appropriate for `education_level` with values {HS, Bachelor's, Master's, PhD}?

- a) One-hot — categories must always be one-hot encoded
- b) Drop the column
- c) Random integers
- d) Ordinal / label encoding preserving the natural order (HS=0, Bachelor's=1, Master's=2, PhD=3); the category order carries information that one-hot encoding throws away
- e) Convert to dates

[☐ Jump to solution](#)

Question 20

When using `pd.get_dummies(...)` to one-hot encode a categorical column inside a **linear regression** model, why is `drop_first=True` often used?

- a) To avoid the dummy-variable trap (perfect multicollinearity). With all K dummies present, the columns sum to 1 and one column is a perfect linear combination of the others, making the design matrix singular for OLS. Dropping one column eliminates the redundancy
- b) To save disk space
- c) To improve accuracy on test data
- d) Scikit-learn requires it
- e) `drop_first=True` is mandatory for tree-based models

[□ Jump to solution](#)

Question 21

A Salesforce dataset has a `lead_company_name` column with 80,000 unique values across 200,000 rows. Why is naive one-hot encoding a problem here?

- a) `pd.get_dummies` cannot handle strings
- b) It changes the target
- c) It explodes the feature matrix to 80,000+ columns, making most algorithms slow and memory-heavy and causing severe sparsity. Use target / mean encoding, frequency encoding, or grouping rare categories into “Other” instead
- d) It always lowers test accuracy
- e) High cardinality is impossible by definition

[□ Jump to solution](#)

Question 22

When is **target (mean) encoding** — replacing each category with the mean of the target for that category — most useful, and what is its main pitfall?

- a) Always; there is no pitfall
- b) Useful for high-cardinality categorical columns where one-hot is impractical. Pitfall: it leaks the target into the feature, so it must be done with cross-validated or out-of-fold means and the encoder must be fit on the training fold only
- c) Only when categories are ordered
- d) Useful only with one-hot encoding
- e) Only for regression problems

[□ Jump to solution](#)

Question 23

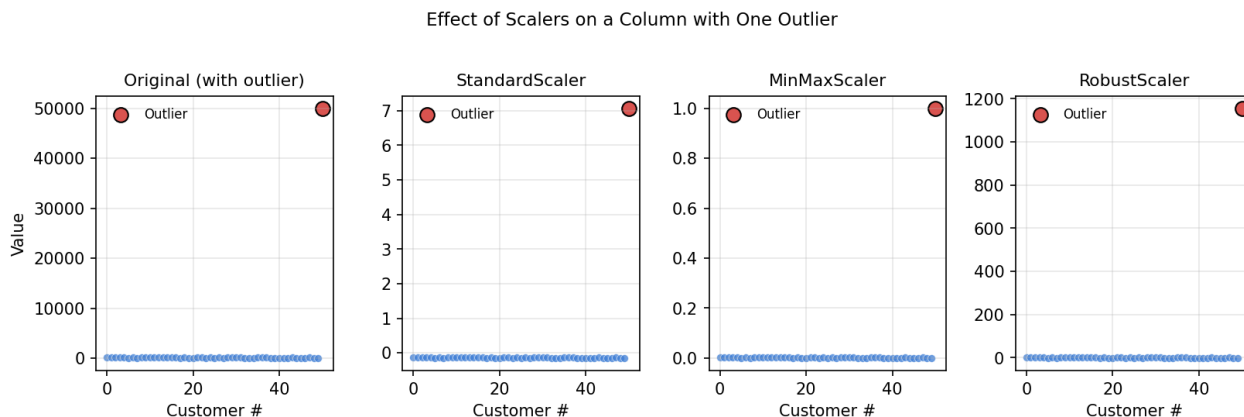
A scikit-learn `OneHotEncoder` is trained on a dataset where `state` only contains the 50 US states. At deployment, a customer enters a new value, "PR" (Puerto Rico). What does `handle_unknown="ignore"` do?

- a) Drops the row entirely
- b) Throws an error and stops the prediction
- c) Adds a new column on the fly during prediction
- d) Encodes the unseen category as all zeros across the dummy columns, allowing the model to keep producing predictions instead of crashing on every new category that appears in production
- e) Replaces "PR" with the most common training category

[□ Jump to solution](#)

Question 24

A team has a feature with one extreme outlier and applies three different scalers. The first panel shows the original column; the next three show how each scaler transforms it.



Which statement correctly distinguishes `StandardScaler` from `MinMaxScaler` (and explains the visible compression in their panels)?

- a) The two produce identical outputs
- b) `StandardScaler` is for classification, `MinMaxScaler` for regression
- c) `MinMaxScaler` removes outliers; `StandardScaler` keeps them
- d) `StandardScaler` centers each feature at zero with unit standard deviation; `MinMaxScaler` rescales each feature to a fixed range (typically $[0, 1]$). `StandardScaler` preserves the shape of the distribution; `MinMaxScaler` is sensitive to outliers (a single huge value squeezes everything else)
- e) `MinMaxScaler` is deprecated

[□ Jump to solution](#)

Question 25

Why does **scaling** matter so much for linear / logistic regression with regularization, K-nearest neighbors, K-means, SVM, and neural networks?

- a) These models cannot read floats
- b) Scaling improves training accuracy by a fixed 10%
- c) They depend on distances or on coefficient magnitudes; without scaling, a feature in the millions (e.g., income) dominates a feature in $[0, 1]$ (e.g., percentage), or the regularization penalty hits some features far harder than others, simply because of unit choice — not because of predictive importance
- d) Scaling is required by Python
- e) Scaling avoids one-hot encoding

[☐ Jump to solution](#)

Question 26

For which algorithm is feature scaling essentially **NOT** necessary?

- a) Logistic regression with L2 regularization
- b) K-Means clustering
- c) Neural networks
- d) Decision Trees and tree-based ensembles (Random Forest, XGBoost) — splits compare values within a single feature, never combine features by distance, so the absolute scale does not matter
- e) SVM with RBF kernel

[☐ Jump to solution](#)

Question 27

A customer-spend column has a few extreme outliers. The team plans to feed it into a regularized linear model and wants to scale it. Which scaler is MOST robust to those outliers?

- a) StandardScaler
- b) MinMaxScaler
- c) Normalizer
- d) RobustScaler — uses the median and the IQR, so a few extreme values do not pull the centering or the scale the way mean and standard deviation would
- e) PolynomialFeatures

[☐ Jump to solution](#)

Question 28

A right-skewed feature `monthly_revenue` has a long tail of large customers. After applying `np.log1p(monthly_revenue)`, the histogram becomes much closer to symmetric. What is the practical benefit for downstream linear models?

- a) The feature becomes categorical
- b) Linear models work better on roughly symmetric, comparably-scaled features. The log transform compresses the long tail, reduces leverage of extreme rows, and often produces a more linear relationship with the target — frequently improving fit and stability
- c) The model gains a free parameter
- d) The model loses interpretability entirely
- e) The feature now has zero variance

[□ Jump to solution](#)

Question 29

A team writes:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, ...)
```

What is the bug?

- a) `StandardScaler` does not exist
- b) The bug is that `random_state` was not specified
- c) `train_test_split` cannot accept scaled data
- d) Scaling before the split lets the test rows contribute to the mean and standard deviation, leaking test information into the training pipeline. Always split first, fit the scaler on training only, and `transform` (not `fit_transform`) the test set
- e) `fit_transform` should be `predict`

[□ Jump to solution](#)

Question 30

A Capital One credit-default model would benefit from a single feature combining `debt` and `income`. Which engineered feature captures the borrower's burden best?

- a) Debt-to-Income ratio (`debt / income`) — a domain-standard ratio that captures relative leverage in one number; far more informative than either component alone
- b) `debt * income`
- c) `debt + income`

- d) `debt - income`
- e) `np.log(debt) + np.log(income)`

[□ Jump to solution](#)

Question 31

You have a column `signup_datetime`. Which set of engineered features would typically be MOST useful for predicting purchase volume?

- a) Use the raw integer Unix timestamp as-is
- b) Convert the timestamp to a string and one-hot encode it
- c) Subtract the mean timestamp and divide by the std
- d) Drop the column — datetimes are not useful
- e) `signup_year`, `signup_month`, `signup_day_of_week`, `signup_hour`, `is_weekend` — turning a single timestamp into multiple cyclical / structural features that the model can exploit

[□ Jump to solution](#)

Question 32

When does **binning** a continuous variable (e.g., grouping age into [`<25`, `25-40`, `40-60`, `60+`]) help a model?

- a) Binning is always harmful
- b) Only for regression problems
- c) It always increases test accuracy
- d) Only when the variable is binary
- e) When the target's relationship with the variable is non-linear and the model is itself linear, binning can capture step-changes the linear model cannot. Tradeoff: bins discard ordering and within-bin variation; tree-based models often do not need it

[□ Jump to solution](#)

Question 33

A Lending Club model already has features `loan_amount` and `annual_income`. The team adds the **interaction** `loan_amount * annual_income`. Why?

- a) Interactions are required by scikit-learn
- b) Interactions remove multicollinearity
- c) Some target relationships depend on combinations — for example, a \$50K loan might be safe at \$200K income but risky at \$30K income. An interaction term lets a linear model express that “the effect of loan size depends on income” rather than treating the two as independent
- d) Interactions are only for categorical data

- e) It saves memory

[□ Jump to solution](#)

Question 34

`PolynomialFeatures(degree=2, include_bias=False)` is applied to a single feature x . What columns does it create?

- a) Only x
- b) The same column repeated 2 times
- c) Only x^2
- d) $\log(x)$ and $\sqrt{x}$
- e) x and x^2 (and if there are multiple input features, also pairwise products)

[□ Jump to solution](#)

Question 35

Why split data into train and test sets BEFORE training the model?

- a) Splitting is optional in scikit-learn
- b) It reduces training time
- c) Splitting is required to encode categorical features
- d) Splitting eliminates the need for cross-validation
- e) Without a held-out test set, the only metric available is training performance — and a flexible model can drive training accuracy to ~ 1 by memorizing the data. The test set is what tells you the model will work on new customers / loans / transactions

[□ Jump to solution](#)

Question 36

A Capital One credit dataset is 5% defaulters and 95% non-defaulters. Why pass `stratify=y` to `train_test_split`?

- a) Stratification preserves the class proportions in BOTH the training and test sets, so the test set's defaulter rate is also $\sim 5\%$ and per-class metrics are stable. Without it, the test set can land at 1% or 9% defaulters by chance, making the metric noisy
- b) Stratification rebalances the classes to 50/50
- c) Stratification is required for regression
- d) Stratification removes the rare class
- e) It runs faster than ordinary sampling

[□ Jump to solution](#)

Question 37

What does `random_state=42` do in `train_test_split`?

- a) Seeds the pseudo-random shuffle that defines the split, so the same code produces the SAME split every run — essential for reproducibility, debugging, and apples-to-apples model comparison
- b) Sets the test size to 42%
- c) Improves test accuracy by ~5%
- d) Forces stratified sampling
- e) Required by scikit-learn or the function will not run

[□ Jump to solution](#)

Question 38

Which test-size choice is most defensible for a 200,000-row dataset?

- a) 0.5 — half the data into the test set so we can evaluate broadly
- b) 0.99 — train on as little as possible
- c) 0.001 — minimize the test
- d) 0.01 — only 0.5% test
- e) 0.20 (or similar; 0.10–0.30 typical) — large enough to give stable test metrics while leaving most of the data for training. With 200K rows, 20% is 40K test points, more than enough for tight confidence intervals on accuracy or AUC

[□ Jump to solution](#)

Question 39

Which definition of **data leakage** is BEST?

- a) The data being too noisy
- b) Information from outside the training set (the test set, the future, or the target itself) inadvertently enters the model during training, producing optimistically biased metrics that collapse when the model meets truly new data
- c) Forgetting to set `random_state`
- d) Logging too many metrics during training
- e) Files corrupted during transfer

[□ Jump to solution](#)

Question 40

A Netflix churn model has a feature `cancellation_email_sent` and gets 99.9% test accuracy. The team is surprised. What is happening?

- a) The model is just very good
- b) Test accuracy of 99.9% is impossible
- c) Target leakage. `cancellation_email_sent` is populated AFTER the customer cancels — at prediction time (when the customer is still active) the value would not be available. The model is essentially predicting “did they cancel” from “did we send the cancellation email,” which is the same column. Drop it (and any other post-cancellation feature)
- d) The model is overfitting
- e) The test set is too small

[□ Jump to solution](#)

Question 41

A team reports unusually high test R^2 on a regression. On inspection, they fit `StandardScaler` on the FULL dataset before splitting. What kind of leakage is this?

- a) Train-test contamination via scaling: the scaler’s mean and standard deviation incorporate test-set values, so the test set is no longer truly held out. Test metrics are mildly inflated. Fix: split first, then `fit_transform` on training and `transform` on test (or use a `Pipeline`)
- b) Multicollinearity
- c) Class imbalance
- d) Underfitting
- e) Stratification leakage

[□ Jump to solution](#)

Question 42

A team imputes missing values with the median of the FULL dataset, then splits. Why is this leakage?

- a) Imputation cannot be done before training
- b) Median is the wrong statistic
- c) The median is computed using both training and test rows; the test rows now slightly influenced the values used to fill the training rows. The fit is on a subtly contaminated training set, and the test metric is optimistically biased. Fit the imputer on training only and apply it to test
- d) The imputer needed to be `IterativeImputer`
- e) The team forgot to set `random_state`

[□ Jump to solution](#)

Question 43

A clean way to combine imputation, scaling, and modeling that AVOIDS leakage during cross-validation is to wrap the steps in a scikit-learn `Pipeline`. Why?

- a) Pipelines run faster
- b) Pipelines reduce memory usage
- c) Pipelines remove the need to set `random_state`
- d) In a pipeline, `cross_val_score` and `GridSearchCV` refit every preprocessing step on each training fold and apply it to the held-out fold. Each fold's "test" portion never touches the imputer or the scaler before scoring — leakage-free by construction
- e) Pipelines automatically pick the best imputer

[☐ Jump to solution](#)

Question 44

A medical-imaging team has 50 patients with 200 scans each (multiple scans per patient). They split rows randomly. What kind of leakage can occur?

- a) Class imbalance leakage
- b) `Random_state` leakage
- c) Group leakage. Some scans of the same patient land in train and others in test. The model can memorize that patient's anatomy and look great on the test set but fail on truly new patients. Fix: use `GroupKFold` or `train_test_split` with patient-level grouping
- d) Stratification leakage
- e) Scaling leakage

[☐ Jump to solution](#)

Question 45

A team adds a feature `customer_was_at_risk_in_previous_quarter` to a churn model. The label `churn` was defined as "did the customer churn in the previous quarter." What is the issue?

- a) The new feature is a deterministic function of the target. The model can recover the target perfectly from this feature. This is target leakage that turns the modeling task into a trivial lookup; remove the feature or redefine it as a true lagged signal that does not encode the target
- b) Nothing — historical features are fine
- c) Data type mismatch
- d) Class imbalance
- e) Multicollinearity with another feature

[☐ Jump to solution](#)

Question 46

An Uber team forecasts ride volume per week using historical data. They split train and test rows RANDOMLY. What is the leakage risk?

- a) Multicollinearity
- b) Class imbalance
- c) Time leakage. With random splitting, future weeks land in the training set and past weeks land in the test set, so the model can use future information to predict the past. Use a time-aware split: train on early weeks, test on later weeks (e.g., `TimeSeriesSplit`)
- d) Underfitting
- e) Stratification leakage

[□ Jump to solution](#)

Question 47

Consider:

```
import pandas as pd
print(df.isna().sum().sort_values(ascending=False).head(5))
```

What does this snippet print?

- a) The five rows with the most missing values
- b) The five most common values in the dataset
- c) Five random rows
- d) The five COLUMNS with the most missing values, ordered from most to fewest
- e) The five most popular categories per column

[□ Jump to solution](#)

Question 48

Consider:

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy="median")
X_train_imp = imputer.fit_transform(X_train)
X_test_imp = imputer.transform(X_test)
```

What is this code doing correctly?

- a) Imputing both train and test with the test-set median

- b) Fitting the imputer on the training set ONLY (computing the per-column median there) and applying the SAME training-set median to the test set with `transform` — exactly the leakage-free pattern
- c) Computing the mean of every column
- d) Encoding categorical columns
- e) Imputing train and dropping test

[□ Jump to solution](#)

Question 49

Consider:

```
df = pd.get_dummies(df, columns=["state", "education"], drop_first=True)
```

What does this code do?

- a) Drops the `state` and `education` columns
- b) Standardizes the columns
- c) Converts numeric columns to strings
- d) Imputes missing values
- e) Replaces `state` and `education` with one-hot dummy columns, omitting one dummy per original column to avoid the dummy-variable trap (multicollinearity in linear models). For 50 states and 4 education levels, that is $49 + 3 = 52$ new columns

[□ Jump to solution](#)

Question 50

Consider:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)
```

Which statement about this code is BEST?

- a) The test set is being fit, not transformed
- b) The order is wrong; `transform` should come before `fit`
- c) `fit_transform` should always be used in one line
- d) `StandardScaler` is incompatible with regression
- e) The scaler learns its mean and standard deviation from `X_train` ONLY (`.fit`), then applies the SAME training-set statistics to both train and test (`.transform`). This is the standard leakage-free pattern

[□ Jump to solution](#)

Question 51

Consider:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

Which statement BEST describes this call?

- a) A leave-one-out split
- b) An 80 / 20 split with no stratification
- c) A 50 / 50 split
- d) An 80 / 20 split with a fixed random seed for reproducibility and stratification on *y*, so the class proportions in *y* are preserved in both splits — the standard recipe for imbalanced classification
- e) A time-series split

[□ Jump to solution](#)

Question 52

Consider:

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000)),
])
pipe.fit(X_train, y_train)
print(pipe.score(X_test, y_test))
```

Why is wrapping these steps in a Pipeline a BEST PRACTICE?

- a) It runs in parallel
- b) The pipeline guarantees a fixed order — impute, then scale, then fit the model — and during cross-validation it refits each step on the training fold only, applying the same fitted transformations to the test fold. Both the imputer and the scaler are leakage-free by construction, and the same recipe applies cleanly at prediction time
- c) It reduces memory
- d) It avoids the need for `train_test_split`
- e) It is required by scikit-learn

[□ Jump to solution](#)

Question 53

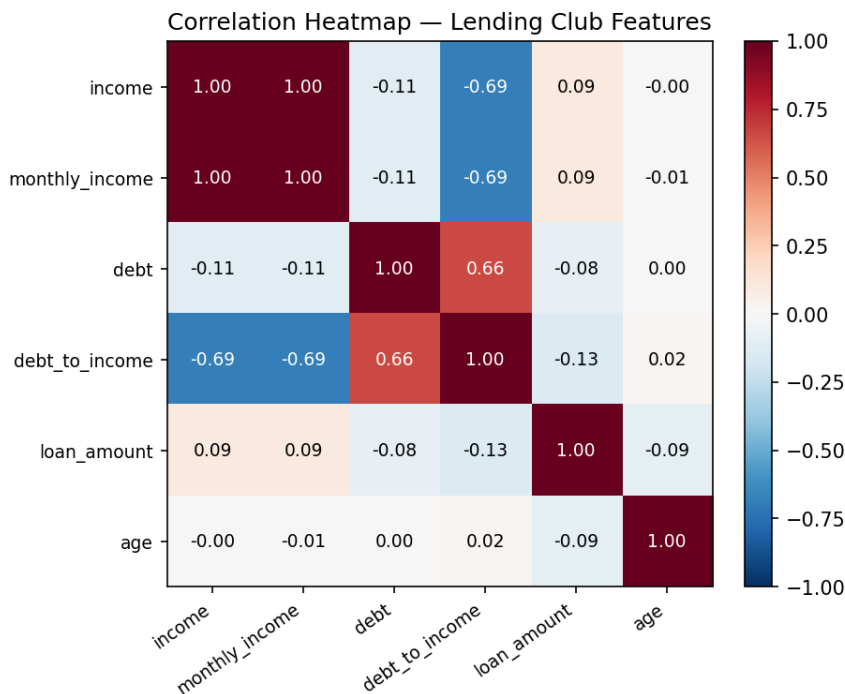
A team produces a `df.isna().mean().sort_values(ascending=False)` bar plot showing the percentage of missing values per column. Three columns sit above 80%, ten columns sit between 5–20%, and the rest are clean. What is the BEST overall plan?

- a) Drop the three high-missingness columns ($\geq 80\%$ missing have too little signal); for the moderate-missingness columns, decide per column based on type (median for skewed numeric, mode for categorical, or KNN/MICE if missingness has structure); always fit imputers on the training set only, ideally inside a Pipeline
- b) Drop every column with any missing values
- c) Run KNN imputation on the entire dataset
- d) Mean-impute every column
- e) Replace all missing values with zero

[□ Jump to solution](#)

Question 54

A team computes the correlation matrix on a Lending Club feature set:



They notice that `income` and `monthly_income` have correlation ≈ 1.00 and `debt_to_income` is highly correlated with `debt`. What is the appropriate response?

- a) Keep both — more features always helps
- b) Run PCA on these two columns
- c) Use one-hot encoding on both

- d) Multiply them together
- e) They are perfectly redundant (one is just the other divided by 1,000). Drop one. Keeping both adds no information and creates singular design matrices that break OLS and degrade regularized linear models

[□ Jump to solution](#)

Question 55

A new analyst at Lending Club proudly reports a model with **train AUC = 0.99** and **test AUC = 0.62**. They followed the standard recipe — but inspection reveals: (i) `StandardScaler` was `fit_transform`-ed on the full dataset before the split, (ii) median imputation was performed on the full dataset before the split, (iii) one feature in the dataset is the post-default `recoveries` amount, and (iv) `total_pymnt` (only known after the loan ends) was kept as a feature. What is the BEST diagnosis and remedy?

- a) Multiple data-leakage problems plus serious post-hoc target features. Fix all four issues simultaneously: split first, then fit the imputer and scaler on training only (use a `Pipeline`); drop `recoveries`, `total_pymnt`, and any other column whose value is only known after the loan outcome. Re-fit and re-evaluate; the inflated train and test numbers will both come down to a realistic and trustworthy level
- b) The model is fine — `train > test` is normal
- c) Add more features
- d) Use a deeper neural network
- e) Just lower the test set size to 5%

[□ Jump to solution](#)

Part 2: Answers and Explanations

Answer 1

Correct: (b). Real-world tabular data rarely arrives in a model-ready state. Missing values, categorical strings, mixed types, wildly different scales, duplicates, and inconsistent encodings all need to be handled before any model can fit. Data preparation is the bridge from raw business data to a numeric matrix — typically the most time-consuming stage of any project.

[□ Back to question](#)

Answer 2

Correct: (e). Data preparation sits between Data Understanding and Modeling in CRISP-DM. Surveys consistently report that 60–80% of a real project's time is spent on this stage.

[□ Back to question](#)

Answer 3

Correct: (b) MNAR. The probability of being missing depends on the unobserved value itself (high earners hide their income). MNAR is the most dangerous case because both deletion and naive imputation produce biased estimates — the model needs domain reasoning or explicit modeling of the missingness mechanism.

[□ Back to question](#)

Answer 4

Correct: (c) `df.isna().sum()`. This returns a per-column count of missing values. Use `.mean()` instead of `.sum()` for the percentage.

[□ Back to question](#)

Answer 5

Correct: (c). With 87% missing, virtually any imputation introduces fabricated values for the vast majority of rows — noise that swamps the small signal. Drop the column. Document the decision so future analysts know it was deliberate.

[□ Back to question](#)

Answer 6

Correct: (e) Median. On right-skewed data the mean is pulled up by the long tail. Imputing with the mean substitutes an unrealistically high “typical” value into many rows. The median is unbiased to skew and is the safe default for skewed numeric features.

[□ Back to question](#)

Answer 7

Correct: (a) Mode. Mean and median are not defined for non-numeric categories. Use the most-frequent value (mode), or treat “missing” as its own explicit category if missingness is informative.

[□ Back to question](#)

Answer 8

Correct: (a) KNN imputation. KNN finds the K most similar non-missing rows on the other features and averages them to fill the missing cell. It is more accurate than median when missingness has structure, but is more expensive and sensitive to feature scaling.

[□ Back to question](#)

Answer 9

Correct: (b). `IterativeImputer` (MICE-style) cycles: model column 1 from columns 2...p, then column 2 from columns 1, 3...p, and so on for several passes. The estimates converge. It is more powerful than KNN when there are systematic relationships between features but adds complexity and computational cost.

[□ Back to question](#)

Answer 10

Correct: (b). A binary missingness indicator preserves the information that the value WAS missing while a separate imputation fills the original numeric column. The model can then learn that missingness itself predicts the target — common in credit and healthcare data.

[□ Back to question](#)

Answer 11

Correct: (b). Fitting the imputer on the full x lets test-set rows contribute to the training-set median. The test set is no longer truly held out and the reported metric is optimistically biased. Fit imputers on training only — typically inside a `Pipeline` so cross-validation handles it automatically.

[☐ Back to question](#)

Answer 12

Correct: (a). KNN and MICE are more accurate when missingness has structure, but they are slower, harder to debug, more sensitive to outliers and scaling, and easier to leak across train/test if not done inside a `Pipeline`. Always weigh the marginal accuracy gain against the operational cost.

[☐ Back to question](#)

Answer 13

Correct: (d) `df.drop_duplicates()`. Use the `subset=[...]` argument to drop based only on certain columns (e.g., the same `order_id`).

[☐ Back to question](#)

Answer 14

Correct: (b). The z-score's denominator (the standard deviation) is itself inflated by extreme values, which can hide most outliers. The IQR uses quartiles, which barely move when a few values are huge. On heavy-tailed business data (revenue, transactions, claim amounts), IQR is the safer detection rule.

[☐ Back to question](#)

Answer 15

Correct: (c). Capping (winsorization) keeps the row's other information while preventing one extreme value from dominating coefficients or distances. It is a reasonable middle ground between dropping the row and leaving the magnitude untouched. Tree-based models rarely need it.

[☐ Back to question](#)

Answer 16

Correct: (d). A log transform compresses long right tails, turns multiplicative relationships into additive ones, and stabilizes variance. Use `np.log1p` (which is $\log(1+x)$) so zeros are well-defined.

[□ Back to question](#)

Answer 17

Correct: (e). Black Friday spikes are real signal, not measurement noise. Dropping or capping them tells the model to ignore one of the most important business patterns in retail. The right move is to keep them and add a feature (`is_black_friday`, holiday indicator, day-of-year) so the model can learn the pattern instead of suppressing it.

[□ Back to question](#)

Answer 18

Correct: (c) One-hot. States have no natural order. Assigning integers 1–50 to them implies a numeric relationship ($CA < NY < TX$) the model would happily exploit. One-hot encoding represents each state with its own binary column, removing the false ordering.

[□ Back to question](#)

Answer 19

Correct: (d) Ordinal / label encoding. Education level has a natural order (HS < Bachelor's < Master's < PhD); preserving that order in a single integer column gives the model usable information. One-hot would work but discards the ordering — a small loss for a model that could exploit it.

[□ Back to question](#)

Answer 20

Correct: (a). With all K dummy columns present, they sum to 1 and the design matrix is rank-deficient. OLS cannot uniquely solve for the coefficients, and regularized models behave oddly. Dropping one dummy makes the others identifiable. Tree-based models are less sensitive to this.

[□ Back to question](#)

Answer 21

Correct: (c). Naive one-hot on a high-cardinality column produces an enormous, sparse matrix that slows training, blows up memory, and pushes most algorithms toward overfitting on rare categories. Common remedies: target / mean encoding (with cross-validated folds), frequency encoding, hashing, or grouping rare categories under "Other".

[□ Back to question](#)

Answer 22

Correct: (b). Target encoding replaces each category with the mean of the target for that category — compact and informative for high-cardinality columns. The major pitfall is leakage: the encoder must be fit on the training fold only, ideally with out-of-fold target means to avoid optimistic bias.

[□ Back to question](#)

Answer 23

Correct: (d). `handle_unknown="ignore"` makes the encoder produce a row of zeros for an unseen category, allowing prediction to proceed. The default behavior (`error`) raises an exception — fine for development, fatal in production where new categories appear constantly.

[□ Back to question](#)

Answer 24

Correct: (d). `StandardScaler` centers at zero with unit variance and preserves the shape of the distribution. `MinMaxScaler` maps to a fixed range — easier to interpret, but a single huge outlier can squeeze almost all other points into a tiny portion of $[0, 1]$. `StandardScaler` is the more common default; `RobustScaler` is the choice when outliers dominate.

[□ Back to question](#)

Answer 25

Correct: (c). All these algorithms compare features either by distance (KNN, K-Means, SVM with RBF) or by penalizing coefficient magnitude (regularized regression). Without scaling, features with naturally large units dominate purely because of unit choice — not because they predict the target better. Scaling levels the playing field.

[□ Back to question](#)

Answer 26

Correct: (d). Trees split a single feature at a threshold; they never combine features by Euclidean distance and never penalize coefficient magnitudes (they have no coefficients). The split point is invariant to monotonic transformations of any feature, so scaling is unnecessary.

[□ Back to question](#)

Answer 27

Correct: (d) RobustScaler. Subtracting the median and dividing by the IQR makes the scaler insensitive to a few extreme values. StandardScaler's mean and standard deviation are pulled by outliers; MinMaxScaler is even more sensitive (one giant value compresses everything else).

Worked example. Imagine six customers with monthly spending of \$50, \$80, \$100, \$150, \$200, and \$50,000 (the last is a "whale"). Each scaler picks a different center and scale. Watch what happens to the FIVE ordinary customers under each:

Scaler	Center	Scale	Scaled values for the five ordinary customers
StandardScaler	mean $\approx$ \$8,430	std $\approx$ \$20,300	all collapse to roughly -0.41 ; the model can no longer distinguish a \$50 spender from a \$200 spender
MinMaxScaler	min = \$50	range = \$49,950	all squashed near zero: 0.000, 0.001, 0.001, 0.002, 0.003
RobustScaler	median $\approx$ \$125	IQR $\approx$ \$90	spread cleanly across $[-1, +1]$: -0.83 , -0.50 , -0.28 , $+0.28$, $+0.83$

The whale stays out at the tail in every case, but only RobustScaler preserves the meaningful variation among the ordinary customers — the variation a regularized linear model needs to see in order to learn anything useful from this feature.

[□ Back to question](#)

Answer 28

Correct: (b). A log-transformed feature has a more symmetric distribution and a more linear relationship with many real-world targets (especially monetary outcomes). Linear and regularized models gain stability and often a noticeable accuracy lift.

[□ Back to question](#)

Answer 29

Correct: (d). `fit_transform(X)` on the entire dataset before splitting lets the test rows contribute to the scaler's mean and standard deviation. The test set is no longer pristine. Fix: split first, then fit the scaler on training and apply it to test — or wrap the scaler in a Pipeline.

[□ Back to question](#)

Answer 30

Correct: (a). Debt-to-income (`debt / income`) is the single most informative engineered feature for credit risk — it captures the borrower's leverage in one number that the lender's risk team uses every day. The other arithmetic combinations are not standard and lose business meaning.

[□ Back to question](#)

Answer 31

Correct: (e). A timestamp is a low-information column on its own; deriving year, month, day-of-week, hour, and a weekend flag exposes the cyclical and structural patterns most predictive models actually exploit. Domain-specific extras (`is_holiday`, `time-of-day-bin`) are worth adding too.

[□ Back to question](#)

Answer 32

Correct: (e). Binning trades smoothness for the ability to capture non-linearities in linear models. For tree-based models the trade is often unfavorable — trees already learn step-changes natively. Bin only when there's a domain reason or a linear model that benefits.

[□ Back to question](#)

Answer 33

Correct: (c). Interactions let a linear model express that the effect of one feature depends on another — exactly the sort of relationship trees learn for free but linear models do not. Add interactions when domain knowledge or EDA suggests them; they double-edge the bias-variance tradeoff and can overfit if added blindly.

[□ Back to question](#)

Answer 34

Correct: (e). With one input feature, `degree=2` produces $[x, x^2]$. With two inputs $[x_1, x_2]$, it produces $[x_1, x_2, x_1^2, x_1 \cdot x_2, x_2^2]$. `include_bias=False` omits the intercept-like 1 column.

[□ Back to question](#)

Answer 35

Correct: (e). A flexible model can drive training accuracy to nearly 1.0 by memorizing the data. The held-out test set is what tells you whether the model generalizes. Splitting is non-negotiable on supervised problems.

[□ Back to question](#)

Answer 36

Correct: (a). Stratification preserves the class proportions in every split. Without it, the test set's class rate can drift far from the population rate by chance, especially on imbalanced problems — making per-class metrics noisy and unreliable.

[□ Back to question](#)

Answer 37

Correct: (a). `random_state` seeds the pseudo-random shuffle used to make the split. Setting it gives the same split every run — essential for reproducibility, debugging, and comparing models without confusing random variation with model improvement.

[□ Back to question](#)

Answer 38

Correct: (e). 0.10–0.30 is the standard range. With 200K rows, 20% gives 40K test points — more than enough for narrow confidence intervals on accuracy or AUC. Halving the data to test would unnecessarily starve training; making the test microscopic gives noisy metrics.

[□ Back to question](#)

Answer 39

Correct: (b). Data leakage is information from outside the training distribution (the test set, the future, or the target itself) entering the model and inflating reported metrics. The hallmark is “great in the lab, bad in production” — leaks usually break the moment the model meets truly fresh data.

[□ Back to question](#)

Answer 40

Correct: (c). `cancellation_email_sent` is populated AFTER cancellation. Including it as a feature means the model is essentially predicting “did they cancel” from “did we send the cancellation email” — a near-deterministic relationship that does not exist at prediction time. Drop the column and any other post-cancellation feature.

[□ Back to question](#)

Answer 41

Correct: (a). Fitting the scaler on the full dataset lets the test rows contribute to the mean and standard deviation. The contamination is small but real — test metrics are mildly inflated. Fix: split first, fit the scaler on training only, use a `Pipeline` so cross-validation refits per fold.

[□ Back to question](#)

Answer 42

Correct: (c). Same logic as scaling-before-split: any statistic computed across the full dataset and used to fill training rows lets the test rows leak into training. Fit imputers on the training set only, with `transform` applied to the test set.

[□ Back to question](#)

Answer 43

Correct: (d). A pipeline groups preprocessing and modeling into a single object. During `cross_val_score` or `GridSearchCV`, scikit-learn refits every step on the training fold and applies it to the held-out fold — leakage-free by construction. The same pipeline also applies cleanly at prediction time, so deployment behavior matches training.

[□ Back to question](#)

Answer 44

Correct: (c) Group leakage. When multiple rows belong to the same entity (patient, customer, transaction session), random splitting can put some of that entity's rows in train and others in test. The model memorizes that entity and looks great on the test rows it has effectively already seen. Use `GroupKFold` / `StratifiedGroupKFold` and provide the group ID.

[□ Back to question](#)

Answer 45

Correct: (a). The new feature is a deterministic function of the target — a special case of target leakage. The model can recover the target trivially, which inflates metrics without learning anything generalizable. Either remove the feature or redefine it as a true lagged signal that does not encode the target itself.

[□ Back to question](#)

Answer 46

Correct: (c) Time leakage. With random splitting, future weeks land in the training set and past weeks land in the test set. The model uses the future to predict the past — easy in the lab, impossible in production. Use `TimeSeriesSplit` (or any split that always trains on earlier data and tests on later data).

[□ Back to question](#)

Answer 47

Correct: (d). `df.isna().sum()` counts missing per column; `.sort_values(ascending=False)` orders them most-to-fewest; `.head(5)` returns the top five. The output is a Series indexed by column name.

[□ Back to question](#)

Answer 48

Correct: (b). `imputer.fit_transform(X_train)` learns the per-column median from training only AND applies it to training. `imputer.transform(X_test)` applies the SAME training-set median to the test set. The test set is never used to compute the median — leakage-free.

[□ Back to question](#)

Answer 49

Correct: (e). `pd.get_dummies` replaces the named categorical columns with one-hot dummy columns; `drop_first=True` omits one dummy per original column to prevent the dummy-variable trap. For 50 states + 4 education levels, that is $49 + 3 = 52$ new binary columns added in place of the two original categorical columns.

[□ Back to question](#)

Answer 50

Correct: (e). This is the leakage-free pattern: fit the scaler on training rows only (so it learns the training mean and std), then apply it to both train and test via `.transform`. The test set never contributes to the scaler's parameters.

[□ Back to question](#)

Answer 51

Correct: (d). 80/20 split because `test_size=0.2`. Reproducible because `random_state=42`. Stratified on `y` so the class proportions are preserved in both splits. This is the standard recipe for imbalanced classification.

[□ Back to question](#)

Answer 52

Correct: (b). A pipeline guarantees a fixed transformation order, refits each step on each training fold during cross-validation (no leakage), and applies the same recipe at prediction time. It also makes hyperparameter tuning across pipeline steps possible with one `GridSearchCV`. This is the "right way" to combine preprocessing and modeling in scikit-learn.

[□ Back to question](#)

Answer 53

Correct: (a). Drop the high-missingness columns (signal too thin); for the moderate-missingness columns choose imputers per column type and per missingness pattern; always fit imputers on training only, ideally inside a `Pipeline` so cross-validation handles the fold-by-fold refitting automatically.

[☐ Back to question](#)

Answer 54

Correct: (e). Correlation of 1.00 means the two columns are deterministic functions of each other (one is the other divided by 1,000). They carry the same information. Keeping both forces a singular design matrix in OLS and degrades regularized linear models with redundant penalty mass. Drop one — and document why.

[☐ Back to question](#)

Answer 55

Correct: (a). Four leakage / post-hoc-feature problems compound here. Splits-after-fitting (scaling and imputation) are the mild end; including post-conclusion features like `recoveries` and `total_pymnt` is the egregious end — they carry the answer directly. Fix all four: split first, wrap imputer + scaler in a `Pipeline` (so cross-validation handles per-fold refitting), and drop every column whose value is only known after the loan ends. The new train AUC will fall from 0.99 and the test AUC will become an honest, deployable number — typically much closer to each other.

[☐ Back to question](#)